

Apprentissage Automatique et réseaux de neurones

Lecture 2: Perceptron

Mona Taghavi

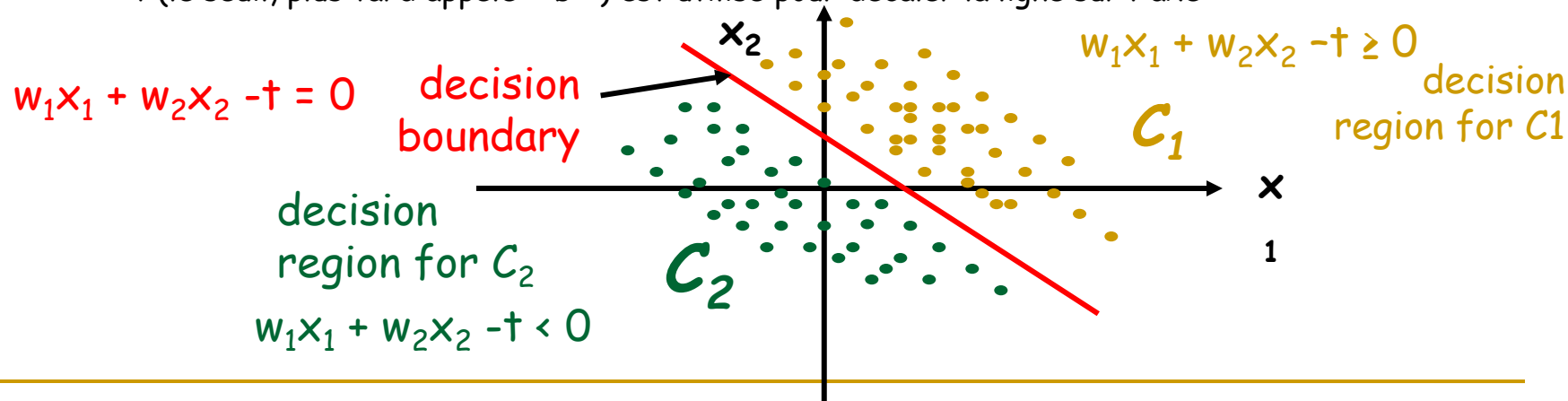
Revision: Example

	Features (x_i)				Output
Student	'A' last year?	Male?	Works hard?	Drinks?	'A' this year?
Richard	Yes	Yes	No	Yes	No
Alan	Yes	Yes	Yes	No	Yes
Alison	No	No	Yes	No	No
Jeff	No	Yes	No	Yes	No
Gail	Yes	No	Yes	Yes	Yes
Simon	No	Yes	Yes	Yes	No

- $(w_1 = 0.2 \ w_2 = 0.1 \ w_3 = 0.25 \ w_4 = 0.1)$
 - Richard: $(1 \times 0.2) + (1 \times 0.1) + (0 \times 0.25) + (1 \times 0.1) = 0.4 < 0.55 \rightarrow$ output is 0 ✓
 - Alan: $(1 \times 0.2) + (1 \times 0.1) + (1 \times 0.25) + (0 \times 0.1) = 0.55 \geq 0.55 \rightarrow$ output is 1 ✓
 - Alison: $(0 \times 0.2) + (0 \times 0.1) + (1 \times 0.25) + (0 \times 0.1) = 0.25 < 0.55 \rightarrow$ output is 0 ✓
 - Jeff: $(0 \times 0.2) + (1 \times 0.1) + (0 \times 0.25) + (1 \times 0.1) = 0.2 < 0.55 \rightarrow$ output is 0 ✓
 - Gail: $(1 \times 0.2) + (0 \times 0.1) + (1 \times 0.25) + (1 \times 0.1) = 0.55 \geq 0.55 \rightarrow$ output is 1 ✓
 - Simon: $(0 \times 0.2) + (1 \times 0.1) + (1 \times 0.25) + (1 \times 0.1) = 0.45 < 0.55 \rightarrow$ output is 0 ✓

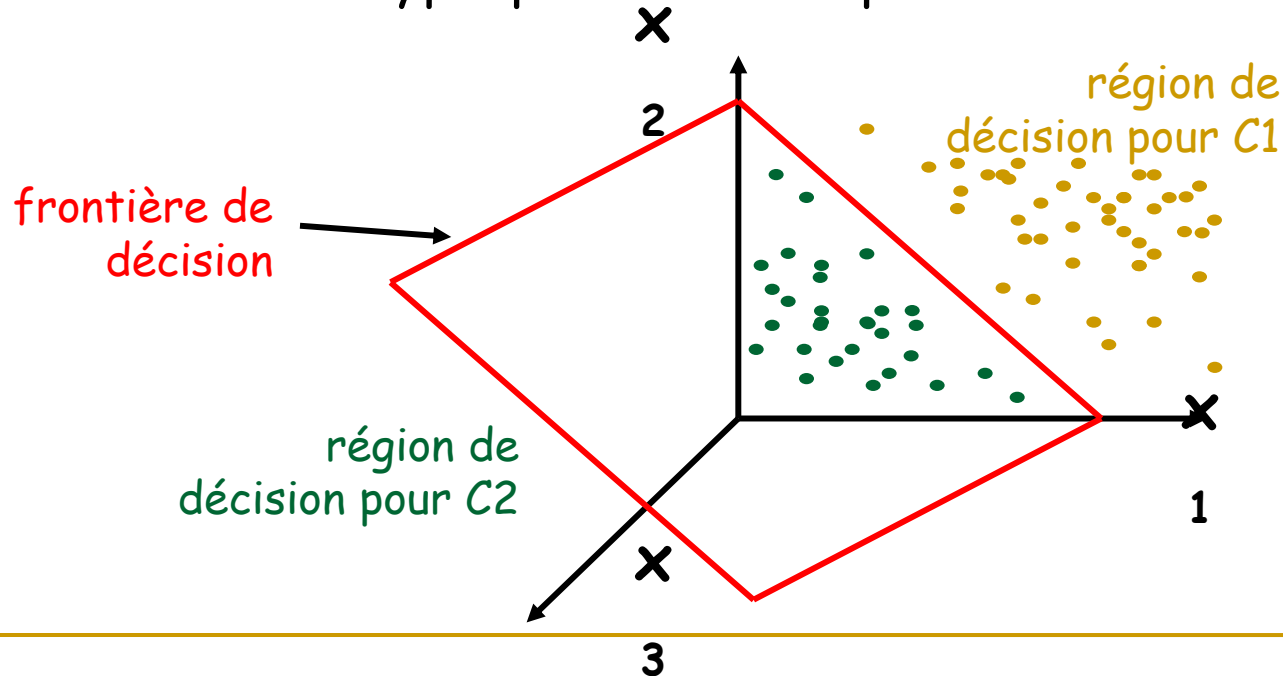
Frontières de décision des perceptrons

- Dans l'exemple précédent, nous avons appris la fonction :
 - Si $(0.2x_1 + 0.1x_2 + 0.25x_3 + 0.1x_4 \geq 0.55)$ alors 1 sinon 0
 - Si $(0.2x_1 + 0.1x_2 + 0.25x_3 + 0.1x_4 - 0.55 \geq 0)$ alors 1 sinon 0
- Supposons que nous n'ayons que 2 caractéristiques :
 - Si $(w_1x_1 + w_2x_2 - t \geq 0)$ alors 1 sinon 0
 - La fonction apprise décrit une ligne dans l'espace d'entrée
 - Cette ligne est utilisée pour séparer les deux classes C_1 et C_2
 - t (le seuil, plus tard appelé « b ») est utilisé pour décaler la ligne sur l'axe



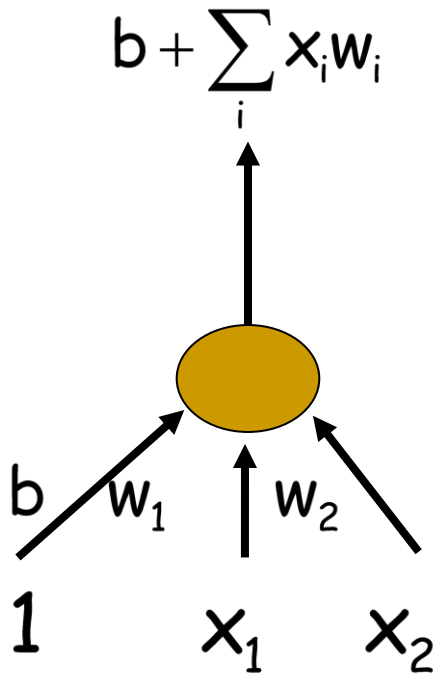
Frontières de décision des perceptrons

- Plus généralement, avec n caractéristiques, la fonction apprise décrit un hyperplan dans l'espace d'entrée.

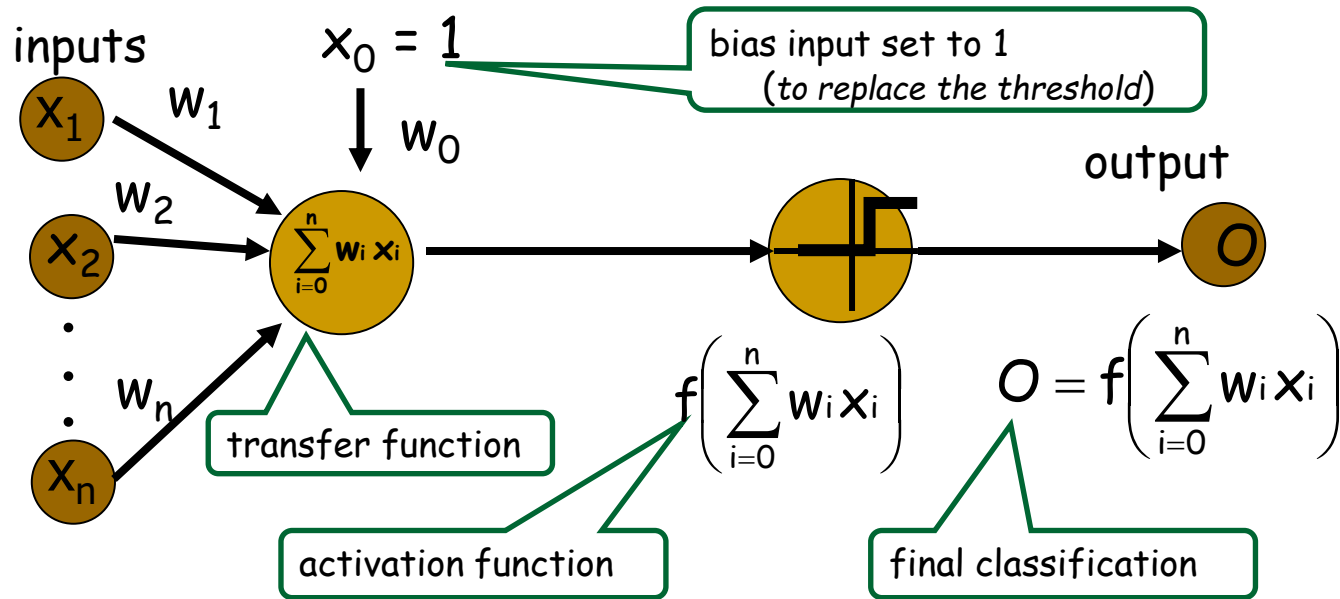


Ajouter un Biais

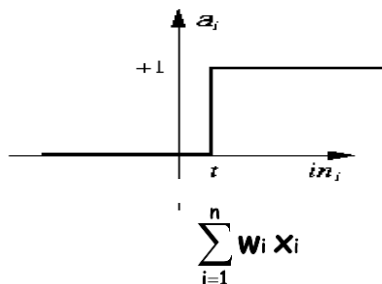
- Nous pouvons éviter d'avoir à déterminer le seuil en utilisant un « biais »
- Un biais équivaut à un poids sur une caractéristique d'entrée supplémentaire qui a toujours une valeur de 1.



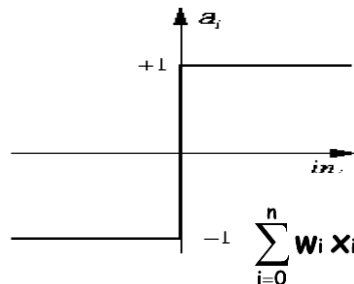
Perceptron - Plus Generalment



Fonctions d'activation courantes



(a) Step function



(b) Sign function

- Fonctions d'activation à seuil brusque :

- échelon (step)
$$O = \begin{cases} 1 & \text{if } \left(\sum_{i=1}^n w_i x_i \right) \geq \tau \\ 0 & \text{otherwise} \end{cases}$$

- signe (sign)
$$O = \begin{cases} +1 & \text{if } \left(\sum_{i=0}^n w_i x_i \right) \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

Taux d'apprentissage

- ❑ Le taux d'apprentissage peut être une valeur constante (comme dans l'exemple précédent)

$$\Delta w = \eta(T - O)$$

Erreur = sortie réelle - sortie cible

Alors :

taux d'apprentissage

- si $T=0$ et $O=1$ (c.-à-d. un faux positif) → diminuer w par η
- si $T=1$ et $O=0$ (c.-à-d. un faux négatif) → augmenter w par η
- si $T=O$ (c.-à-d. pas d'erreur) → ne pas changer w

valeur de la caractéristique d'entrée x_i

- ❑ Ou, une fraction de la caractéristique d'entrée x_i

$$\Delta w_i = \eta(T - O) x_i$$

Ainsi, la mise à jour est proportionnelle à la valeur de x

- si $T=0$ et $O=1$ (c.-à-d. un faux positif) → diminuer w_i par ηx_i
- si $T=1$ et $O=0$ (c.-à-d. un faux négatif) → augmenter w_i par ηx_i
- si $T=O$ (c.-à-d. pas d'erreur) → ne pas changer w_i

C'est ce qu'on appelle la **règle delta** ou **règle d'apprentissage du perceptron**

Théorème de convergence du perceptron

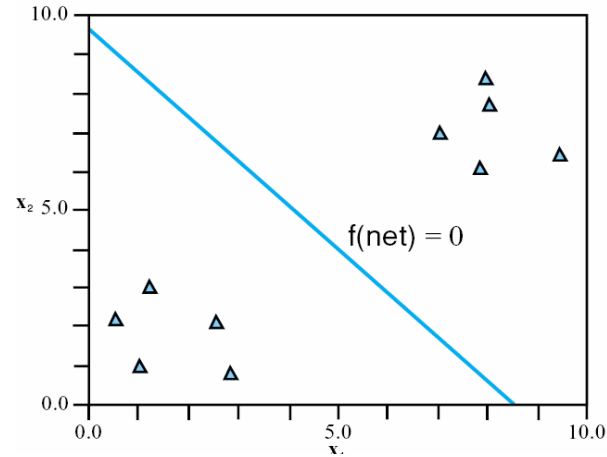
- Parcourir l'ensemble des exemples d'apprentissage.
- Supposons qu'une solution avec une erreur nulle existe.
- La règle delta trouvera une solution en un temps fini.

Exemple de la règle delta

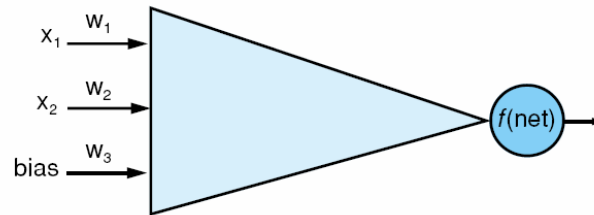
- données d'apprentissage :

x_1	x_2	Output
1.0	1.0	1
9.4	6.4	-1
2.5	2.1	1
8.0	7.7	-1
0.5	2.2	1
7.9	8.4	-1
7.0	7.0	-1
2.8	0.8	1
1.2	3.0	1
7.8	6.1	-1

- graphique des données d'apprentissage :



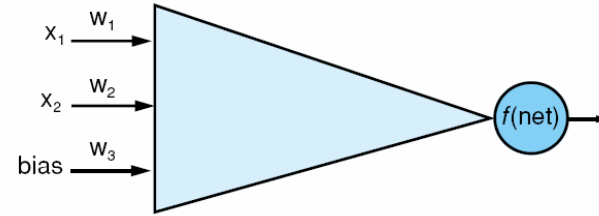
- perceptron



Entraînons le Perceptron

- supposons une initialisation aléatoire

- $w_1 = 0.75$
- $w_2 = 0.5$
- $w_3 = -0.6$



- Supposons :
 - fonction signe (seuil = 0)
 - taux d'apprentissage $\eta = 0.2$

Entraînement

- donnée #1: $f(0.75 \times 1 + 0.5 \times 1 - 0.6 \times 1) = f(0.65) \rightarrow 1 \checkmark$
- donnée #2: $f(0.75 \times 9.4 + 0.5 \times 6.4 - 0.6 \times 1) = f(9.65) \rightarrow 1 \square$

○ $\rightarrow \text{erreur} = (-1 - 1) = -2$

$\rightarrow w1 = w1 - 2 \times 0.2 \times 9.4$

$= 0.75 - 3.76 = -3.01$

$\rightarrow w2 = w2 - 2 \times 0.2 \times 6.4 = -2.06$

$\rightarrow w3 = w3 - 2 \times 0.2 \times 1 = -1.00$

donnée #3: $f(-3.01 \times 2.5 - 2.06 \times 2.1 - 1 \times 1) = f(-12.84) \rightarrow -1 \square$

- $\rightarrow \text{erreur} = (1 - -1) = 2$

$\rightarrow w1 = -3.01 + 2 \times 0.2 \times 2.5 = -2.01$

$\rightarrow w2 = -2.06 + 2 \times 0.2 \times 2.1 = -1.22$

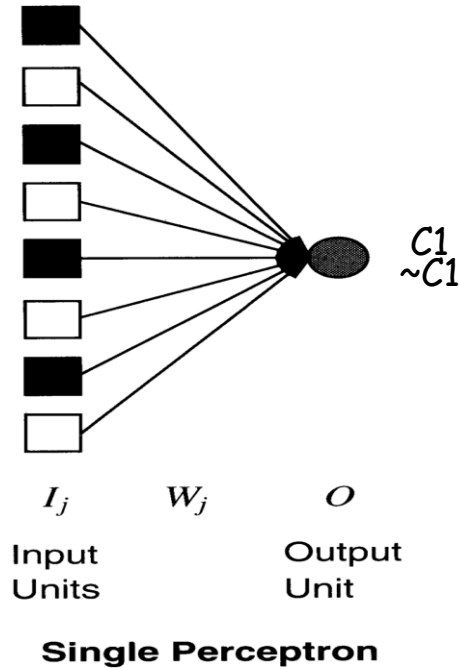
$\rightarrow w3 = -1.00 + 2 \times 0.2 \times 1 = -0.60$

- répéter... après 500 itérations, nous convergeons vers :

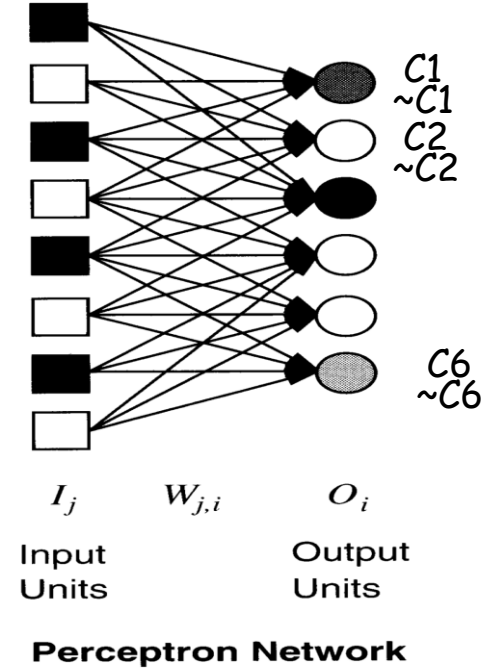
$w1 = -1.3 \quad w2 = -1.1 \quad w3 = 10.9$

x_1	x_2	Output
1.0	1.0	1
9.4	6.4	-1
2.5	2.1	1
8.0	7.7	-1
0.5	2.2	1

Un réseau de Perceptrons



- Jusqu'à présent, nous avons étudié un perceptron unique
- Mais si la sortie doit apprendre plus qu'une décision binaire (oui/non)
- Ex : apprendre à reconnaître un chiffre --> 10 sorties possibles --> nécessite un réseau de perceptrons



Limites du Perceptron

- En 1969, Minsky et Papert ont montré formellement quelles fonctions pouvaient ou non être représentées par des perceptrons
- Seules les fonctions linéairement séparables peuvent être représentées par un perceptron

Dartmouth Conference: The Founding Fathers of AI



John McCarthy



Marvin Minsky



Claude Shannon



Ray Solomonoff



Alan Newell

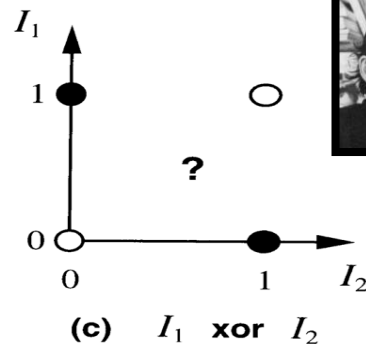
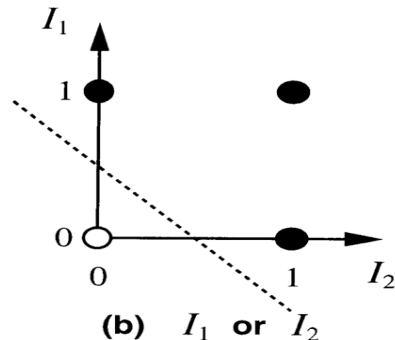
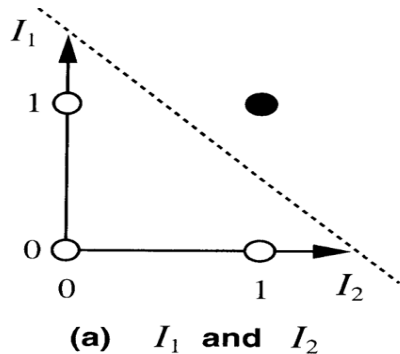


Herbert Simon

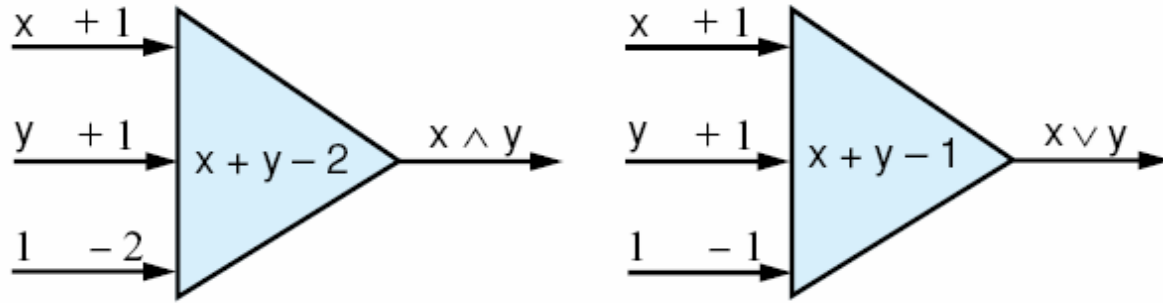


Arthur Samuel

And three others...
Oliver Selfridge
(Pandemonium theory)
Nathaniel Rochester
(IBM, designed 701)
Trenchard More
(Natural Deduction)



Perceptrons AND et OR



x	y	$x + y - 2$	Output
1	1	0	1
1	0	-1	-1
0	1	-1	-1
0	0	-2	-1

Exemple : la fonction XOR

- On ne peut pas construire un perceptron pour apprendre la fonction OU-exclusif

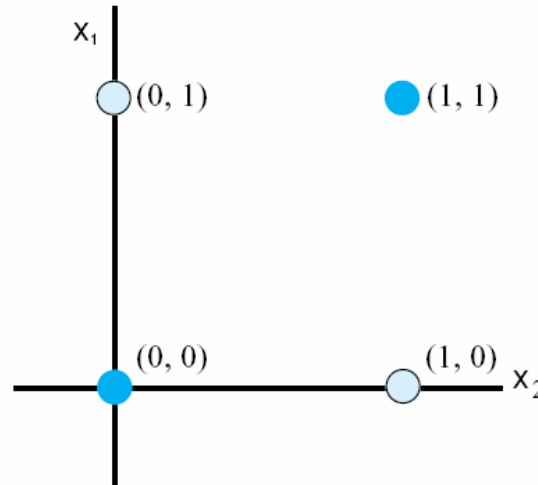
- Pour apprendre XOR, avec :
 - deux entrées x_1 et x_2
 - deux poids w_1 et w_2
 - un seuil $\dagger$

x_1	x_2	Sortie
1	1	0
1	0	1
0	1	1
0	0	0

- c'est-à-dire, doit avoir :
 - $(1 \times w_1) + (1 \times w_2) < \dagger$ (pour la 1ère ligne)
 - $(1 \times w_1) + 0 \geq \dagger$
 - $0 + (1 \times w_2) \geq \dagger$
 - $0 + 0 < \dagger$
- Ce qui n'a pas de solution... donc un perceptron ne peut pas apprendre XOR

La fonction XOR - Visuellement

- Dans un espace bidimensionnel (2 caractéristiques pour X)
- Aucune ligne droite en deux dimensions ne peut séparer
 - $(0, 1)$ et $(1, 0)$ de
 - $(0, 0)$ et $(1, 1)$.



Fonctions non linéairement séparables

- Les problèmes du monde réel ne peuvent pas toujours être représentés par des fonctions linéairement séparables...
- Cela a entraîné une baisse de l'intérêt pour les réseaux de neurones dans les années 1970

Linear vs. nonlinear problems

